Adapting Mamba Models for Deployment on Microcontrollers

Enabling Linear-Time Sequence Modeling on Ultra-Low-Power Edge Devices

Bartosz Drabiński[1]

Supervisor: Bo Yang[1]
Responsible Professor: Qing Wang[1]

[1]**EEMCS, Delft University of Technology, The Netherlands**

A Thesis Submitted to EEMCS Faculty Delft University of Technology,
In Partial Fulfilment of the Requirements
For the Bachelor of Computer Science and Engineering
June 17, 2026

Name of the student: Bartosz Drabiński
Final project course: CSE3000 Research Project
Thesis committee: dr. Qing Wang, Mark Neerincx

Abstract

As machine learning expands into diverse domains, TinyML has emerged as a crucial paradigm for deploying models on highly resource-constrained microcontrollers, which typically feature less than 256 KB of RAM and 240 MHz CPUs. However, executing complex mathematical operations on these devices remains a significant challenge, necessitating novel model designs and hardware-aware optimization. The Mamba architecture, built around State-Space Models, is a promising candidate due to its compact parameterization and strong performance on long-context tasks. Nevertheless, Mamba was originally designed for highly parallelized GPUs, making its adaptation for TinyML non-trivial. This paper evaluates Mamba deployment strategies on microcontrollers using TensorFlow Lite Micro. We propose architecture modifications and optimization techniques tailored specifically to microcontroller constraints. Our deployment of a quantized Mamba model achieves a 60.4 KB peak RAM footprint on a Keyword Spotting task—a 74% memory reduction compared to prior work (MambaLite-Micro). Furthermore, we analyze the trade-offs of quantization, demonstrating that while it substantially reduces memory, it can introduce latency overhead on hardware lacking acceleration of INT8 operations. To mitigate code size and loop-unrolling overheads, we introduce a model-splitting technique that enables the execution of larger models. Our findings demonstrate that while Mamba is a viable architecture for TinyML, further research is required to fully optimize SSM implementations for edge hardware.

Keywords: TinyML, Mamba, microcontroller, Edge AI

1 Introduction

Machine learning has recently enabled solutions to problems that were difficult or infeasible with traditional algorithms. This progress produced both large, general-purpose models (e.g., large language models) and many compact, task-specific models.

Most deployments still rely on centralized data centres, which are efficient but impose limitations: they require network connectivity, add latency, and raise privacy concerns.

Edge AI reduces these limitations by running inference locally on user devices. TinyML [1] focuses on the extreme end of this spectrum, designing models that run on microcontrollers (MCUs). MCUs are severely resource-constrained: typical devices provide one or two CPU cores (usually clocked below 240 MHz) and on the order of 512 KB of RAM, so they cannot host large models or their full-precision weights. Low latency is also essential because many TinyML applications (for example, audio or video streams) require real-time processing.

Numerous model families have been developed for different trade-offs, including multilayer perceptrons (MLPs), convolutional neural networks (CNNs), recurrent neural networks (RNNs), and Transformers. The recently proposed Mamba architecture [2] has emerged as a promising alternative to Transformers for long-context tasks. Mamba combines ideas from CNNs and RNNs and leverages Selective State-Space Model (S6) block to achieve a smaller memory footprint while maintaining strong performance on long sequences.

Prior work has explored optimizations for Mamba such as quantization [3], pruning, and knowledge distillation [4]. MambaLite-Micro [5] implemented PyTorch-style inference for Mamba directly in C and ran it on ESP32-S3 and STM32 microcontrollers; this preserves compatibility but makes architecture changes and further optimizations cumbersome.

This paper addresses the following overarching research question: *How can Mamba architectures be adapted and optimized for efficient inference on resource-constrained microcontrollers?* To address this comprehensively, we formulate three specific sub-questions:

- **RQ1:** What are the primary memory and computational bottlenecks when deploying Mamba models on microcontrollers?
- **RQ2:** How do standard quantization strategies affect Mamba's inference latency, memory footprint, and task accuracy on these edge devices?
- **RQ3:** To what extent can hardware-unfriendly components of the Mamba architecture (e.g., specific activation functions) be simplified or replaced to fit microcontroller constraints, and what are the resulting trade-offs in performance?

To answer these questions, we profile deployment strategies on actual hardware, isolate the underlying hardware bottlenecks, and evaluate the efficacy of translating typically GPU-centric optimizations to microcontroller architectures.

The remainder of this paper is organized as follows. Section 2 surveys related work and foundational background. Section 3 details our proposed optimizations and architectural modifications, while Section 4 outlines how the experiments were conducted. Experimental results and a subsequent discussion are presented in Section 5. Finally, Section 6 addresses ethical considerations, and Section 7 concludes the paper with directions for future work.

2 Related Work and Background

This section reviews TinyML constraints and recent Mamba work relevant to MCU deployment. We summarize common TinyML optimizations (quantization, pruning, distillation), then focus on Mamba's SSM-based design and prior deployment attempts that motivate our contributions.

2.1 TinyML

TinyML concerns the design, optimization, and deployment of machine learning models on highly resource-constrained embedded devices such as microcontrollers. Models for these devices are typically trained with the target constraints in mind and then optimized for inference, since typical devices often have less than 256 KB of Data RAM (DRAM).

These constraints limit which architectures are practical. Common choices include convolutional neural networks (CNNs) and recurrent neural networks (RNNs), which can be more parameter-efficient than multilayer perceptrons (MLPs) [6].

MCUs have limited computational power, so latency is a critical concern. Many TinyML applications (for example, audio or accelerometer streams) require real-time processing, which makes models that rely on expensive mathematical operations harder to deploy. This is especially true for activation functions using trigonometric terms (e.g., Tanh) or exponentiation (e.g., Softmax, SiLU) [7].

The limited or slow floating-point support on many MCUs makes quantization an important optimization strategy. Quantization reduces the precision of weights and activations from 32-bit floats to lower-bit formats (for example, 8-bit or 4-bit integers), which can substantially reduce model size and memory usage. These gains can come at the cost of accuracy, and memory savings are not always proportional due to format and graph overheads [8]. In some cases, careless quantization can lead to large accuracy drops, especially for models not designed with quantization in mind [3].

Other strategies include pruning, knowledge distillation, and low-rank factorization [9]. Pruning removes redundant parameters, while knowledge distillation transfers knowledge from a larger teacher model into a smaller student. These techniques can be effective but are often more complex to apply, and reported size reductions vary by method and task [4].

2.2 Mamba architecture

The Mamba architecture [2] is a recent proposal that blends convolutional and recurrent design ideas to capture long-range dependencies while keeping a smaller memory footprint than Transformers. A top-level diagram of a Mamba block appears in Figure 1.

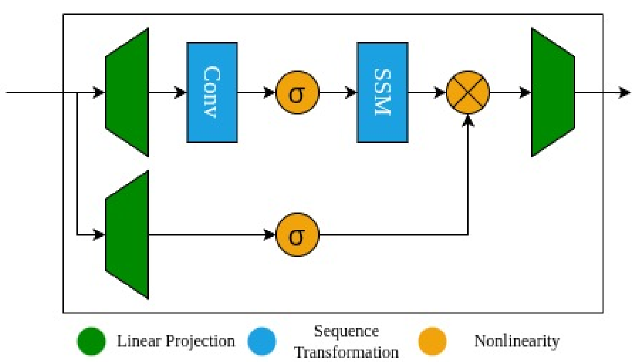


Figure 1: Diagram of the Mamba block

Mamba's main building block is a Selective State-Space Model (S6) layer, which captures long-range structure while avoiding the quadratic complexity of attention. The S6 formulation can be written as

$$h_t = \overline{A}h_{t-1} + \overline{B}x_t$$
$$y_t = Ch_t + Dx_t$$

In Mamba the S6 matrices are conditioned on the input, allowing the dynamics to adapt over time. S6 layers can be implemented using efficient scan (parallel-prefix) algorithms that parallelize well on modern hardware, but their recurrent-style structure can make them sensitive to quantization and numerical issues on deeply constrained hardware [3; 10].

Most prior work reducing Mamba model sizes targets mobile or embedded GPUs rather than MCUs. For example, Physics-Guided Tiny-Mamba Transformer [11] is reduced to 0.5M parameters for Jetson Nano, Tiny-ViM [12] targets 5M parameters for vision, and FEMBA [10] applies aggressive quantization at 7.8M parameters. These models remain far larger than typical MCU budgets.

Some efforts (e.g., LightMamba [13]) explore Selective State-Space Models for resource-constrained tasks such as ultra-wideband positioning, but demonstrations on actual microcontrollers are limited. MambaLite-Micro [5] implements PyTorch-style inference for Mamba in C and runs on ESP32-S3 and STM32 MCUs, preserving compatibility but making architecture changes and further optimizations more cumbersome. Overall, deployment tooling for Mamba on MCUs remains immature.

2.3 Research gap

Two primary gaps motivate this work. First, many TinyML optimization techniques exist, but their applicability to Mamba is not well understood on MCU-class hardware. Second, while MambaLite-Micro demonstrates feasibility, it does not explore architecture modifications or deployment strategies that could further reduce memory usage and latency on MCUs. This paper evaluates deployment strategies, identifies Mamba-specific bottlenecks on microcontrollers, and proposes targeted improvements.

3 Proposed Mamba-MCU Optimizations

This section presents the deployment choices and architecture adaptations designed to fit Mamba on microcontrollers. We first justify the chosen toolchain and hardware, then describe the model reimplementation and the optimizations evaluated (quantization, looped SSM, activation simplifications).

3.1 Embedded Deployment

Deployment of machine learning models on microcontrollers, while feasible, still presents challenges. The MambaLite-Micro authors chose a path that preserves exact PyTorch inference results by extracting weights and hardcoding them in C. This approach preserves compatibility but makes architecture changes and further optimizations cumbersome, since any model modification requires C changes.

For this paper we decided not to take that route and instead selected an existing model-porting tool. We investigated frameworks such as TensorFlow Lite for Microcontrollers (TFLM, also referred to as LiteRT) [14] and ExecuTorch [15]. The hardware-agnostic workflow of TFLM provided more flexibility for experimentation, better documentation, and fewer deployment issues. By using TFLM we avoid additional intermediary formats (for example, we do not convert to ONNX), removing conversion differences as a factor in the experiments.

There are several tools built into and around TFLM. We use 'tflm_model_transforms' to strip redundant and debug data from the model, enabling fairer comparisons of model sizes by reducing graph overhead. We also use the AI Edge Quantizer, which provides precise control over quantization and operates directly on TFLite model files. The different deployment options are illustrated in Figure 2.

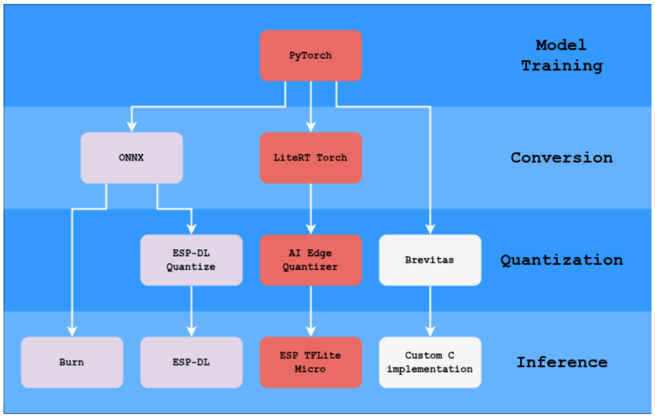


Figure 2: Diagram of the deployment steps and some options for a TinyML

We also considered other deployment options such as Brevitas [16] (used by FEMBA) and TorchAO [17], which support quantization on PyTorch models and Quantization-Aware Training (QAT). Brevitas can target very low bit-widths. We did not use these tools in the final implementation because QAT often requires kernel fusion that is difficult to achieve efficiently on microcontrollers.

Due to time constraints we implemented and tested models only on the ESP32 (Figure 3) family of microcontrollers (ESP32 and ESP32-S3) using 'esp-tflite-micro', a vendor-specific implementation of TFLM. We avoided vendor-specific pipelines such as ESP-DL or STM32Cube.AI so results would not be bound to a single MCU platform.

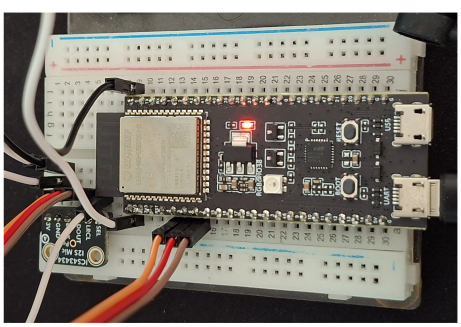


Figure 3: Photo of an ESP32S3 with ICS43434 microphone module attached

3.2 Model Architecture

We do not explore extensive hyperparameter search in this work; instead, we adopt the MambaLite-Micro architecture [5] so comparisons remain meaningful and our effort can focus on deployment and optimization. The architecture comprises a fully connected input layer, a single Mamba block, an average-pooling layer, and a fully connected output layer. The Mamba block input size is 64 with an expansion ratio of 2.

Because the original Mamba implementation relies on custom GPU kernels not available in TFLM, we re-implemented the architecture in PyTorch. This gives a more flexible deployment pipeline and makes it easier to modify activations and reduce expensive exponentiation operations prior to conversion.

3.3 Deployment Optimization

For deployment optimization we focus primarily on quantization, evaluating INT8 weight and activation quantization.

Because TFLM lacks a general looping mechanism, the SSM implementation is unrolled by default, causing model size to grow linearly with sequence length. This is problematic for longer sequences (for example, the Speech Commands dataset). To mitigate this, we split the deployment model into three parts: pre-SSM, SSM step, and post-SSM. The inference code loops over the SSM step, reusing the same weights for each time step and significantly reducing model size. We compare both unrolled and looped implementations on the HAR dataset to study trade-offs.

4 Experimental Setup and Evaluation

This section describes datasets, preprocessing, evaluation metrics, and measurement procedures used to quantify accuracy, peak RAM, and latency. We provide enough detail to allow replication of conversion, quantization, and microcontroller benchmarking steps.

4.1 Dataset and Preprocessing

Many datasets have been used and demonstrated as viable for TinyML applications; these typically use sensor data such as audio, images, or accelerometer signals. For this work we use two datasets: the Human Activity Recognition (HAR) dataset [18] and the Google Speech Commands dataset [19].

These datasets are popular in the TinyML community and represent different data modalities (accelerometer and audio), which enables comparison with prior work such as MambaLite-Micro [5].

The HAR dataset consists of recordings of volunteers performing activities of daily living while carrying a waist-mounted smartphone with embedded inertial sensors. The data is preprocessed into 561 features. Following previous work, we pad this input to 570 features and split them into 10 time steps of 57 features each, so that it matches the format used in prior work. State-of-the-art models can achieve nearly 98% accuracy on this dataset [20].

For the Google Speech Commands dataset we preprocess audio into Mel-frequency cepstral coefficients (MFCCs). We extract 40 MFCC features per time step, producing 51 time steps with 40 features each. While some implementations reduce the number of classes from 35 to 10, we kept all 35 classes to exercise long-sequence behavior. State-of-the-art models can achieve around 96% accuracy on this dataset [21].

4.2 Model Training and Conversion

The experimental design evaluates parts of the deployment pipeline: model architecture, optimization strategies, and deployment choices. We measure both peak memory usage and latency, which are critical for TinyML.

For both datasets we use hyperparameters: Hidden size: 64; Expansion ratio: 2; State size: 16; Batch size: 32; Learning rate: 0.002 (drop to 0.001 after 10 epochs); Epochs: 20

The size of the model was chosen to be comparable to the MambaLite-Micro implementation. We do not perform extensive hyperparameter search, since our focus is on deployment and optimization rather than maximizing accuracy.

The model is trained using the Adam optimizer with a learning rate of 0.002 and a step learning rate scheduler that reduces the learning rate by half every 10 epochs. We train for 20 epochs, which is sufficient for convergence on these datasets.

We train the model in PyTorch and then convert it to TFLite using the TFLM converter. Then we apply quntization using the AI Edge Quantizer We only evaluate static post-training quantization. We do this with 2000 samples from the training sets of the two datasets. We found that increasing the number of samples further did not significantly change quantization results.

4.3 Evaluation Metrics and Measurement Procedures

Measuring peak memory and latency on an embedded device is challenging, but TFLM provides tools to help. For memory usage we use the TFLM memory recorder to measure peak memory and to separate permanently allocated versus temporary buffers. For latency we adapted the default TFLM profiler to measure total inference time and time per operation type, which helps identify bottlenecks and the impact of optimizations.

For accuracy testing we run the full test set on the development PC for both the PyTorch implementation and after conversion to TFLite to ensure conversion does not introduce significant degradation. We also measure accuracy after quantization to quantify its impact.

For testing on the microcontroller we chose the ESP32 (ESP32-WROOM-32), ESP32S3 (ESP32-S3-DevKitC-1-N32R16V). This allows us to focus on one hardware family while still comparing a more basic model (ESP32) to a more powerful one (ESP32-S3). It also allows us to compare with MambaLite-Micro, which also uses ESP32-S3.

On the microcontroller we evaluate accuracy on a small subset (50 samples) from each test set due to resource constraints, which provides a sanity check that the final embedded implementation matches expected behavior.

The code for training, conversion, and microcontroller benchmarking is available at https://github.com/drabart/MCUFriendlyMamba.

5 Results and Discussion

This section presents experimental results for accuracy, model sizes, latency, and memory usage, followed by interpretation and discussion of practical trade-offs when deploying Mamba on MCUs.

5.1 Model accuracy

We first trained models on a development PC using a custom PyTorch implementation of the Mamba block. Training produced validation accuracies of 98.6% for HAR and 90.09% for the KWS task.

We then converted the models to TFLite and evaluated accuracy before and after quantization. Results are shown in Table 1 for both the original (full) model and the split model.

Table 1: Accuracy results for full and split models.

Dataset	Float32	Int8
HAR PyTorch	93.89%	-
HAR TFLite Full	93.89%	93.59%
HAR TFLite Split	93.89%	92.37%
KWS PyTorch	88.96%	-
KWS TFLite Full	88.96%	85.06%
KWS TFLite Split	88.96%	84.10%

We also measured output file sizes after each optimization step (Table 2). Raw file size can be misleading for formats that include graph metadata in addition to raw weights: quantize/dequantize operators and preserved names can outweigh tensor-size savings. Using ‘tflm_model_transforms‘ to strip debug and name metadata reduced file size by up to 30% in our experiments.

Table 2: Model sizes before and after quantization and optimization.

Dataset	Original	Quantized	Optimized
HAR Full	195 KB	102 KB	70.5 KB
HAR Split*	160 KB	62 KB	50.9 KB
KWS Full	319 KB	267 KB	–**
KWS Split*	165 KB	63 KB	51.9 KB

*Sizes of the 3 partial models have been added up

**Concatenation operation too long to handle optimization.

It is often assumed that storing model data in flash is much slower than storing it in RAM. We measured a simple array-sum benchmark with data in DRAM versus flash on the ESP32 and found flash access 2.15x slower for this task (far smaller than typical desktop disparities). Note that TFLM copies relevant arrays from ROM to RAM on tensor initialization, so this initial-copy overhead is separate from steady-state inference behavior.

5.2 Embedded Performance

The next important metrics are the inference time (latency) and RAM usage on the microcontroller.

We started by assessing the impact each operation had on the latency (Figure 7). This motivated us to limit usage of exponentiation in the inference, as it was the easiest to eliminate, and this optimization has already been implemented by similar works [22]. This led us to create 3 similar Mamba models: *float noopt*, *float*, *float opt*. The noopt model does not replace any of the exp operations, the float model replaces the SiLU and Softmax operations with ReLU, and the float opt in addition to the function replacement replaced `A_bar = torch.exp(delta_t * A_reshaped)` line with Euler Discretization `A_bar = 1.0 + delta_t * A_reshaped`. The latency of those models can be seen on Figure 4. We also measured accuracy of those models, which can be seen in Table 3. Because of the similar accuracy for the *float noopt* and *float* models, as well as the noticeable accuracy decrease for the quantized *float opt* model, we decided to use *float* as the baseline for future optimizations[1].

Table 3: Accuracy results for float HAR models with varying amount of exponentiation removed.

Model	Float32	Int8
float noopt	94.06%	93.42%
float	94.64%	94.67%
float opt	93.76%	89.96%

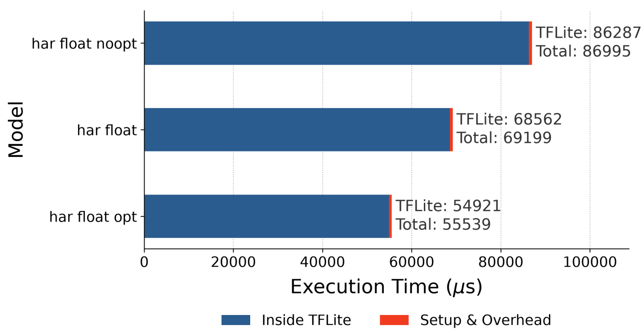


Figure 4: Comparison of different amounts of exponentiation being eliminated from the float HAR model

Although quantized models are often faster due to reduced floating-point work, for our TFLM Mamba models the INT8 variants are slower (Figure 5). INT8 latency is over nearly two times larger than float. This is a surprising result, as on the Figure 7 we can see the MUL operation dominating the chart for INT8 inference, but MUL is one of the few operations that ESP32S3 has hardware acceleration implemented for. We also include an INT8 ANSI run latency, as we discovered that for HAR dataset specifically, enabling ESP_NN optimizations causes the on device measured accuracy to drop from 96% to 56%. We only encountered this issue for this specific model on this dataset, so all the following results are using ESP_NN optimizations.

Next we measured the latency of the proposed splitting operation in comparison to the standard single model approach (Figure 6). We find that our splitting solution introduces a lot of overhead. While the TFLM part of latency also increases, that increase is much less drastic.

[1]We later found out that depending on the seed the accuracy decrease for the *float opt* might not occur, and achieved 94.81%, 94.71% accuracy results on one testing run

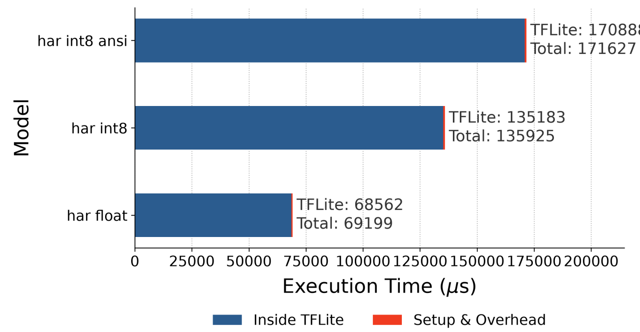


Figure 5: Graph of the latency of the HAR models

Memory usage behaves as expected: INT8 variants require significantly less RAM (Figure 8). We also see that the model split even though it needs additional buffers to transfer data between models uses less memory than the non split versions. The size of the buffers in relation to the ones used for inference also can be different based on the input shape (Figure 9)

5.3 Discussion

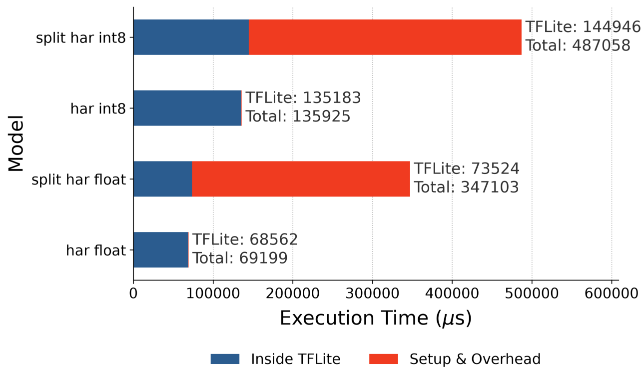
Our findings demonstrate that post-training quantization is a viable strategy for compact Mamba models intended for edge deployment. The resulting accuracy trade-offs are highly problem-dependent; as noted in existing literature like *MambaQuant*, quantizing State Space Models (SSMs) presents distinct numerical challenges. This complexity is evident in our KWS models, which exhibit an accuracy loss of nearly 4 percentage points under INT8 quantization. Nonetheless, this level of degradation remains acceptable within the constraints of many practical application domains.

Model segmentation provides a mechanism to deploy larger, more complex models that would otherwise fail execution constraints. Empirically, the unified KWS architecture could not be executed without this splitting step. This limitation stems primarily from structural constraints within TFLM, where the native concatenation operator restricts merging to a maximum of 10 concurrent vectors, whereas the KWS model demands the concatenation of 51 distinct timesteps.

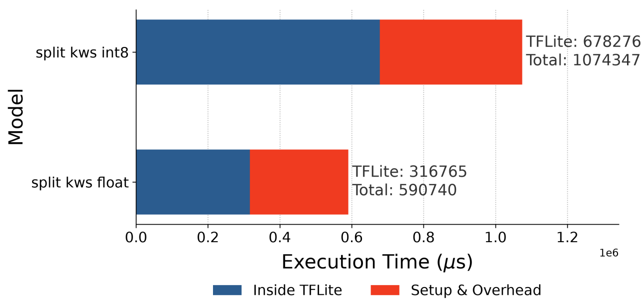
An unexpected benefit of model segmentation was the reduction in peak working memory. We hypothesize that this behavior is tied to TFLM's memory allocation framework, which relies on a greedy heuristic; segmenting the network appears to partition the allocation problem into sub-problems that allow the heuristic to make globally superior layout decisions. If this hypothesis holds, memory reduction may be highly sensitive to network topology and may not uniformly occur across all architectures.

However, our empirical data shows that split models exhibit heightened sensitivity to quantization-induced accuracy drops. We attribute this vulnerability to two primary factors:

1. Static quantization algorithms can compute a more optimal global calibration fit when applied to a fully unrolled computational graph.
2. Numerical quantization errors compound more aggressively within our iterative, manual loop implementation



(a) Latency comparison between the HAR models



(b) Latency comparison between KWS float32 and int8 models

Figure 6: Operation-type latency breakdown for the full models.

of the SSM recurrence.

Because an unrolled graph treats each recurrent step as a distinct layer, it can dynamically assign unique scaling factors and zero-points to separate instances of identical mathematical operations. This granular optimization explains why the unrolled, full model structure preserves higher fidelity under static quantization.

While our deployed Mamba configurations are significantly smaller than state-of-the-art large language models, they achieve compelling task-specific performance: nearly 94% accuracy on HAR and approximately 89% on KWS within our evaluation frameworks. This underscores Mamba's viability for compact, sequential edge processing.

In terms of execution profiles, a large portion of inference latency is concentrated in the basic arithmetic operations (`SUM`, `MUL`, `ADD`) and the exponential functions (`EXP`) driving the SSM layer. In its native formulation, the SSM layer maps poorly to standard TFLM runtime loops due to the computational tax of computing exponentials on bare-metal MCUs.

Developing a specialized, natively quantized TFLM kernel dedicated to the SSM primitive would likely provide substantial latency reductions. However, project timelines precluded the realization of a custom kernel. Furthermore, while model segmentation expands the upper bound of deployable model sizes, it introduces a severe latency penalty that must be factored into real-time system designs.

5.4 Limitations

Several limitations qualify the findings of this study. First, the software tooling and compiler infrastructure required to compile and deploy Mamba architectures onto microcontrollers remain highly immature, and tight development timelines constrained our ability to fully explore alternative deployment pathways.

Second, our empirical evaluation was restricted to two widely utilized TinyML benchmarks. Consequently, these workloads do not capture performance across other domains, such as computer vision or video processing, where Mamba's linear scaling properties could offer distinct advantages.

Third, our analysis evaluated only FLOAT32 and INT8 data types. Exploring ultra-low bit-width quantization (e.g., INT4 or sub-byte formats) represents a valuable research direction, but such exploration is currently bottlenecked by the restricted operator support available in mainstream TFLM releases.

Fourth, hardware benchmarking was confined exclusively to a single microcontroller ecosystem (the Espressif ESP32 series). Because microcontrollers vary significantly in their proprietary hardware acceleration blocks, deploying these models on alternative vendor architectures could yield vastly different performance and latency profiles.

This hardware dependency was further highlighted by stability issues; during our evaluation, we discovered bugs within the vendor-specific optimization libraries for certain layers, which introduces an element of platform-specific unreliability into our reported data.

Finally, our model-splitting approach introduces operational complexity by requiring the inference code to be highly model-aware, manually orchestrating data across sub-graphs. This design drastically inflates the overall inference latency. While implementing the recurrent SSM loop as a unified, custom-quantized TFLM kernel would maintain deployment simplicity and achieve similar memory-saving trade-offs, we leave the development of such specialized kernels to future work.

6 Responsible Research

This section outlines the ethical considerations, reproducibility frameworks, and the structured use of generative AI tools throughout this study. Specifically, we document dataset and code compliance, experimental replication protocols, and potential dual-use risks alongside corresponding mitigation strategies.

6.1 Ethical Considerations

While this study does not involve human subjects or direct third-party participants, the broader implications of deploying machine learning on edge devices warrant careful ethical consideration. By optimizing machine learning frameworks for resource-constrained microcontrollers, this work shifts computational workloads from centralized datacenters to low-power edge devices, thereby contributing to environmental sustainability via reduced operational carbon footprints.

Furthermore, on-device processing inherently enhances user privacy by keeping raw data local and minimizing

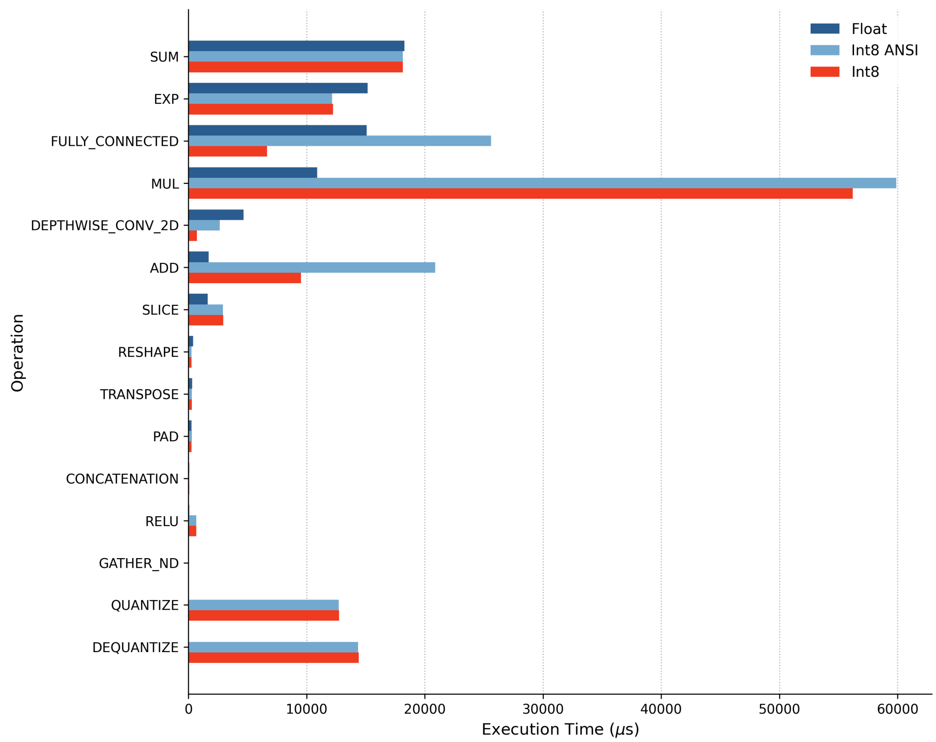


Figure 7: Comparison of operation impacts on latency of the Float, INT8 ANSI, and INT8 HAR models

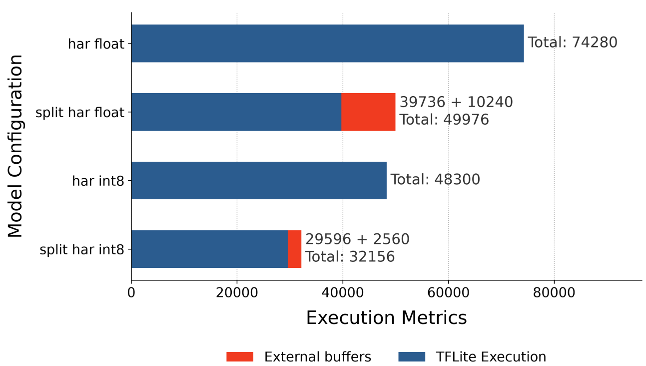


Figure 8: Graph of peak RAM usage of the HAR models

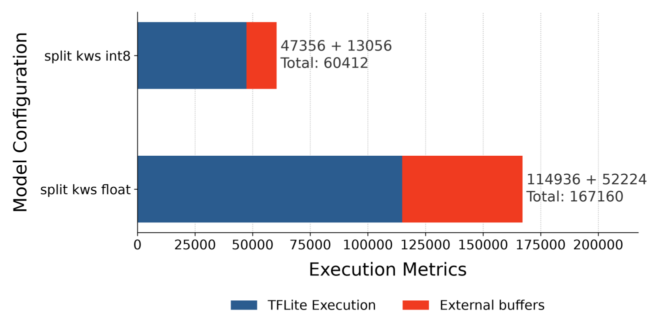


Figure 9: Graph of peak RAM usage of the KWS models

cloud-based data exposure. However, decentralized processing also introduces potential risks, such as facilitating covert local surveillance. Consequently, downstream deployments must integrate robust hardware safeguards, operational transparency, and appropriate governance frameworks. Because TinyML applications frequently process highly sensitive telemetry (e.g., biomedical or acoustic data), developers utilizing this framework must implement stringent security measures and maintain transparency regarding data lifecycle management.

Additionally, the proliferation of specialized edge devices introduces e-waste risks due to rapid hardware obsolescence. To mitigate this impact, practitioners are encouraged to prioritize hardware longevity, provide long-term software maintenance, and adhere to responsible electronic waste disposal practices.

Finally, we acknowledge the systematic risk of algorithmic and dataset bias. The datasets benchmarked in this work (HAR and Google Speech Commands) are established standards within the TinyML community; however, they may exhibit representation gaps across diverse populations or operational environments. These limitations must be factored into the generalization and interpretation of the empirical results.

6.2 Reproducibility

All datasets utilized in this research are publicly available, and all associated software dependencies are governed by open-source licenses permitting research usage. To ensure reproducibility, we have comprehensively documented the experimental pipeline, including model training, optimization, quantization, and microcontroller benchmarking. While full replication requires specific hardware targets, the ESP32 family utilized in this study is widely accessible within the research community.

To ensure empirical fidelity, performance profiling was conducted using a unified, consistent profiler across all experimental iterations, with results systematically annotated by the specific microcontroller variant used. The complete codebase—including replication scripts, library dependencies, and exact version constraints—is hosted in a public repository to facilitate community verification.

6.3 Use of AI Tools

Generative AI tools were utilized strictly as supportive instruments during the development and drafting phases of this research. AI assistance was restricted to writing utility scripts, debugging software implementations, and refining the grammatical clarity of the manuscript.

Specifically, AI tools were leveraged for the following tasks:

- Automating boilerplate scripts for the experimental pipeline execution. Debugging syntax and integration issues within the experimental codebase.
- Enhancing the syntactic clarity, flow, and structure of the written narrative.
- Assisting with automated LaTeX formatting and structural consistency.

All core scientific decisions, experimental designs, and data interpretations were executed independently by the authors. Generative AI tools were not employed to synthesize, alter, or influence experimental outcomes, empirical data, or scientific conclusions.

7 Conclusions and Future Work

This paper examined how Mamba can be adapted and optimized for inference on microcontrollers. Our experiments support three concise findings:

- Deploying via TFLM (with model-graph stripping) substantially reduces peak RAM compared to the C-based MambaLite-Micro workflow (roughly a 75% reduction in our runs).
- Memory vs. latency trade-off: INT8 quantization cuts peak RAM by about $\approx$ 33% and model file size by $\approx$ 60%, but on the tested ESP32-class hardware it increased latency because efficient INT8 vector instructions are lacking.
- Split vs. full deployment: splitting into pre-SSM / SSM-step / post-SSM lowers peak RAM and permits longer sequences, but increases sensitivity to static quantization and complicates the runtime pipeline.

Given our findings, the TFLM-based deployment pipeline seems to be a better path for Mamba on microcontrollers than the C-based approach of MambaLite-Micro, as it allows for more flexible experimentation and even better memory efficiency. While the C-based approach could allow for even more aggressive optimizations they would be hard to deploy for general use models. To make the deployment more practical the model split that we implemented would need to be integrated into the TFLM as a custom operator.

Taken together, Mamba is a strong candidate for TinyML on MCUs when memory optimizations are prioritized, but real-world latency depends heavily on hardware and kernel support. A next step that could be pursued is implementing a custom, quantized Selective State-Space Model kernel for TFLM and running targeted microbenchmarks across MCU families to guide low-level optimizations. The model split that we applied was applied in order to mitigate the lack of a looping mechanism in TFLM, but it had a side effect of reducing the peak RAM usage, the reason fo this could be further investigated. We also recommend systematically evaluating pruning methods (structured and unstructured) to reduce parameter count and memory footprint, and combining those experiments with Quantization-Aware Training and lower-bit formats.

References

[1] P. Warden and D. Situnayake, *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. "O'Reilly Media, Inc.".

[2] A. Gu and T. Dao, "Mamba: Linear-Time Sequence Modeling with Selective State Spaces."

[3] Z. Xu, Y. Yue, X. Hu, Z. Yuan, Z. Jiang, Z. Chen, J. Yu, C. Xu, S. Zhou, and D. Yang, "MambaQuant: Quantizing the Mamba Family with Variance Aligned Rotation Methods."

[4] I. F. Shihab, S. Akter, and A. Sharma, "Efficient Unstructured Pruning of Mamba State-Space Models for Resource-Constrained Environments," *arXiv preprint*, 2025.

[5] H. Xu, J. Xia, W. Yang, Y. Sui, and S. Xia, "MambaLite-Micro: Memory-Optimized Mamba Inference on MCUs."

[6] M. Tri Lê, P. Wolinski, and J. Arbel, "Efficient Neural Networks for Tiny Machine Learning: A Comprehensive Review," vol. 17, no. 4, pp. 1–41.

[7] T. Cassimon, S. Vanneste, S. Bosmans, S. Mercelis, and P. Hellinckx, "Designing resource-constrained neural networks using neural architecture search targeting embedded devices," vol. 12, p. 100234.

[8] K. Dokic, M. Martinovic, and D. Mandusic, "Inference speed and quantisation of neural networks with TensorFlow Lite for Microcontrollers framework," in *2020 5th South-East Europe Design Automation, Computer Engineering, Computer Networks and Social Media Conference (SEEDA-CECNSM)*, pp. 1–6.

[9] T. Suwannaphong, F. Jovan, I. Craddock, and R. McConville, "Optimising TinyML with quantization and distillation of transformer and mamba models for indoor localisation on edge devices," vol. 15, no. 1, p. 10081.

[10] A. Tegon, N. Lehmann, Y. Li, A. Cossettini, L. Benini, and T. M. Ingolfsson, "FEMBA on the Edge: Physiologically-Aware Pre-Training, Quantization, and Deployment of a Bidirectional Mamba EEG Foundation Model on an Ultra-low Power Microcontroller."

[11] C. Li, D. Huang, K. Yao, X. Ni, L. Shen, and F. Luo, "Physics-Guided Tiny-Mamba Transformer for Reliability-Aware Early Fault Warning."

[12] X. Ma, Z. Ni, and X. Chen, "TinyViM: Frequency Decoupling for Tiny Hybrid Vision Mamba."

[13] T. Wang, Y. Shu, L. Qiao, D. Ding, and G. Li, "Light-Mamba: A Resource-Efficient Deep-Learning Method for UWB NLOS Identification Based on Selective State-Space Modeling," vol. 13, no. 5, pp. 8735–8748.

[14] R. David, J. Duke, A. Jain, V. J. Reddi, N. Jeffries, J. Li, N. Kreeger, I. Nappier, M. Natraj, S. Regev, R. Rhodes, T. Wang, and P. Warden, "TensorFlow Lite Micro: Embedded Machine Learning on TinyML Systems."

[15] M. Nachin, D. Desai, S. S. Jia, C. Lai, M. Liu, J. Szwejbka, R. Alvarez, R. Ascani, D. Bort, M. Candales, *et al.*, "ExecuTorch - a unified PyTorch solution to run AI models on-device," *arXiv preprint arXiv:2605.08195*, 2026.

[16] G. Franco, A. Pappalardo, and N. J. Fraser, "Xilinx/brevitas," 2025.

[17] A. Or, A. Jain, D. Vega-Myhre, J. Cai, C. D. Hernandez, Z. Zheng, D. Guessous, V. Kuznetsov, C. Puhrsch, M. Saroufim, S. Rao, T. Tran, and A. Samardžić, "TorchAO: PyTorch-Native Training-to-Serving Model Optimization."

[18] D. Anguita, A. Ghio, L. Oneto, X. Parra, and J. L. Reyes-Ortiz, "A Public Domain Dataset for Human Activity Recognition Using Smartphones,"

[19] P. Warden, "Speech Commands: A Dataset for Limited-Vocabulary Speech Recognition," *ArXiv e-prints*, Apr. 2018.

[20] Y. Enokibori, "rTsfNet: A DNN model with Multi-head 3D Rotation and Time Series Feature Extraction for IMU-based Human Activity Recognition."

[21] J. Wang, L. Yu, L. Huang, C. Zhou, H. Zhang, Z. Song, Z. Ma, and Z. Zhang, "Efficient Speech Command Recognition Leveraging Spiking Neural Network and Curriculum Learning-based Knowledge Distillation,"

[22] J. Kim, J. Lee, J. Lin, A. Kanani, M. Sun, U. Y. Ogras, and J. Park, "eMamba: Efficient Acceleration Framework for Mamba Models in Edge Computing."